

# MRI Foreground/Background Detection

Amir R. Sadri  
ars329@case.edu  
December 16, 2025

Abstract

This note outlines how we derive foreground and background masks for MRI volumes in RadQy. Each section documents one detection method, its assumptions, and the clean-up steps needed so FG/BG pixels are reproducible across datasets. sarvar

Contents

|     |                                                     |   |
|-----|-----------------------------------------------------|---|
| 1   | Introduction                                        | 2 |
| 2   | Image Processing Segmenters                         | 2 |
| 2.1 | OtsuHull (OtsuHull.py)                              | 2 |
| 2.2 | Adaptive Border Segmentation (AdaptiveBorderSeg.py) | 2 |
| 2.3 | Hybrid Otsu + Convex Hull (HybridOtsuHullSeg.py)    | 3 |
| 3   | Deep Learning Segmenters                            | 3 |
| 3.1 | U-Net                                               | 3 |

# 1 Introduction

This document lists the foreground/background (FG/BG) segmentation methods in `backend/seg/mri`. Each section mirrors a Python module and notes its inputs, thresholds, morphology, and fallbacks so downstream IQMs get consistent masks, matching the FG/BG logic relied on in MRQy [1].

Unless noted, we normalize each MRI slice to uint8 with dynamic range [0, 255] before thresholding. Inputs outside this range are clipped, and all downstream FG/BG masks are therefore binary and non-negative. Any negative-valued inputs are treated as invalid and skipped from analysis.

## 2 Image Processing Segmenters

Classical pipelines based on intensity statistics and morphology.

### 2.1 OtsuHull (OtsuHull1.py)

Legacy variant mirroring the original hybrid Otsu pipeline with more defensive fallbacks (see Sadri et al., 2020 MRQy) and rooted in classic Otsu thresholding [2]. This subsection is listed first because it matches the earliest implementation shipped with RadQy.

**Algorithm 1:** OtsuHull foreground/background estimation

1. Input slice  $I$ ; cast to uint16.
2. Histogram-equalize  $I$  to get  $H$ .
3. Otsu on  $I$  and  $H$  to obtain masks  $m_1, m_2$  and FG fractions  $w_1, w_2$ .
4. Hybrid recombination:  $S = w_1 \cdot I + w_2 \cdot H$  (normalize to image range).
5. Otsu on  $S$  to get final mask  $M$ .
6. Cleanup: convex hull of  $M$  (fallback: hole filling if hull fails).
7. Outputs:  $fg = M, bg = 1 - M$ .

Notes on the weights and masks:  $m_1$  is the Otsu mask on the raw slice  $I$ ,  $m_2$  is the Otsu mask on the hist-equalized slice  $H$ . The weights  $w_1 = |m_1|/|I|$  and  $w_2 = |m_2|/|I|$  measure how much of each slice is classified as foreground; they scale the raw and equalized images when forming  $S = w_1 \cdot I + w_2 \cdot H$  so that the cleaner mask (smaller foreground fraction) influences the recombined image more.

### 2.2 Adaptive Border Segmentation (AdaptiveBorderSeg.py)

Single-slice segmenter that estimates background from the image border and keeps the largest non-background component.

---

**Algorithm 2:** AdaptiveBorder foreground/background estimation

---

1. Normalize input slice to uint8 via 1st/99th percentile scaling.
  2. Extract border ring (default 12 px each side); compute  $\mu$ ,  $\sigma$  on non-zero border pixels (fallback: full image if border empty).
  3. Mark background where  $I < \mu - 1\sigma$ .
  4. Morphology: binary closing then opening with  $5 \times 5$  kernel; fill holes.
  5. Foreground = inverse of background; keep largest connected component.
  6. Safety: if FG area < 5%, set FG to all-ones (entire slice).
  7. Outputs:  $(fg, bg)$  with  $bg = 1 - fg$ .
- 

## 2.3 Hybrid Otsu + Convex Hull (HybridOtsuHullSeg.py)

Modern hybrid approach combining Otsu on raw and histogram-equalized images with convex hull cleanup.

---

**Algorithm 3:** Hybrid Otsu + Convex Hull

---

1. Robust-scale input slice to  $[0, 1]$  via 1st/99th percentiles; cast to uint8 for thresholding.
  2. Compute Otsu masks on raw ( $x$ ) and hist-equalized ( $h$ ) images to get FG fractions  $w_1, w_2$ .
  3. Recombine:  $S = w_1 \cdot x + w_2 \cdot h$ .
  4. Otsu on  $S$  to obtain mask  $M$ .
  5. Cleanup: apply convex hull to  $M$  (fallback: hole filling if hull unavailable).
  6. Outputs:  $(fg, bg)$  with  $fg = M$ ,  $bg = 1 - fg$ .
- 

## 3 Deep Learning Segmenters

Learned models for FG/BG inference.

### 3.1 U-Net

Learned U-Net inference [3] using the default checkpoint at `backend/seg/mri/models/unet.pt`. The samples generated by `tests/sample_random_seg_masks.py` are saved under `docs/tex/Figure/MRI/seg/unet`. [1]

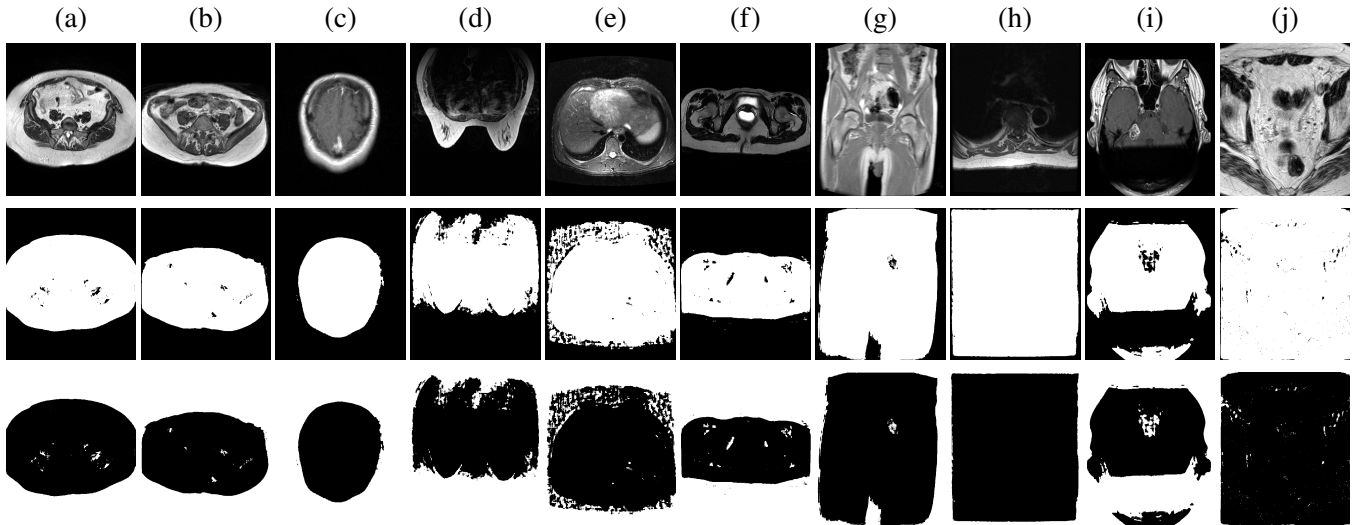


Figure 1: U-Net FG/BG detection MRI samples. Row 1: original MRI slices; Row 2: foreground masks; Row 3: background masks. Columns: (a) Pelvis T2 AX, (b) Cervix T1 AX, (c) Brain T1 Post AX, (d) Breast T1 AX, (e) Abdomen T1 FSE AX, (f) Abdomen T2 FSE AX, (g) Pelvis GRE FLASH AX, (h) Chest T1 AX, (i) Ear T1 TSE AX, (j) Bladder T1 AX.

## References

- [1] MRQy—An open-source tool for quality control of MR imaging data. *Medical physics*, 2020.
- [2] A multi-scale 3D Otsu thresholding algorithm for medical image segmentation. *Digital Signal Processing*, 2017.
- [3] U-net: Convolutional networks for biomedical image segmentation. *International Conference on Medical image computing and computer-assisted intervention*, 2015.